



IA_PCOM7E January 2026

Individual Presentation



I. Project Premise & Requirements

A. Project Premise

The premise of the project is an automated agent-driven content moderation system for a social media platform that permits users to upload image files.

B. Requirements

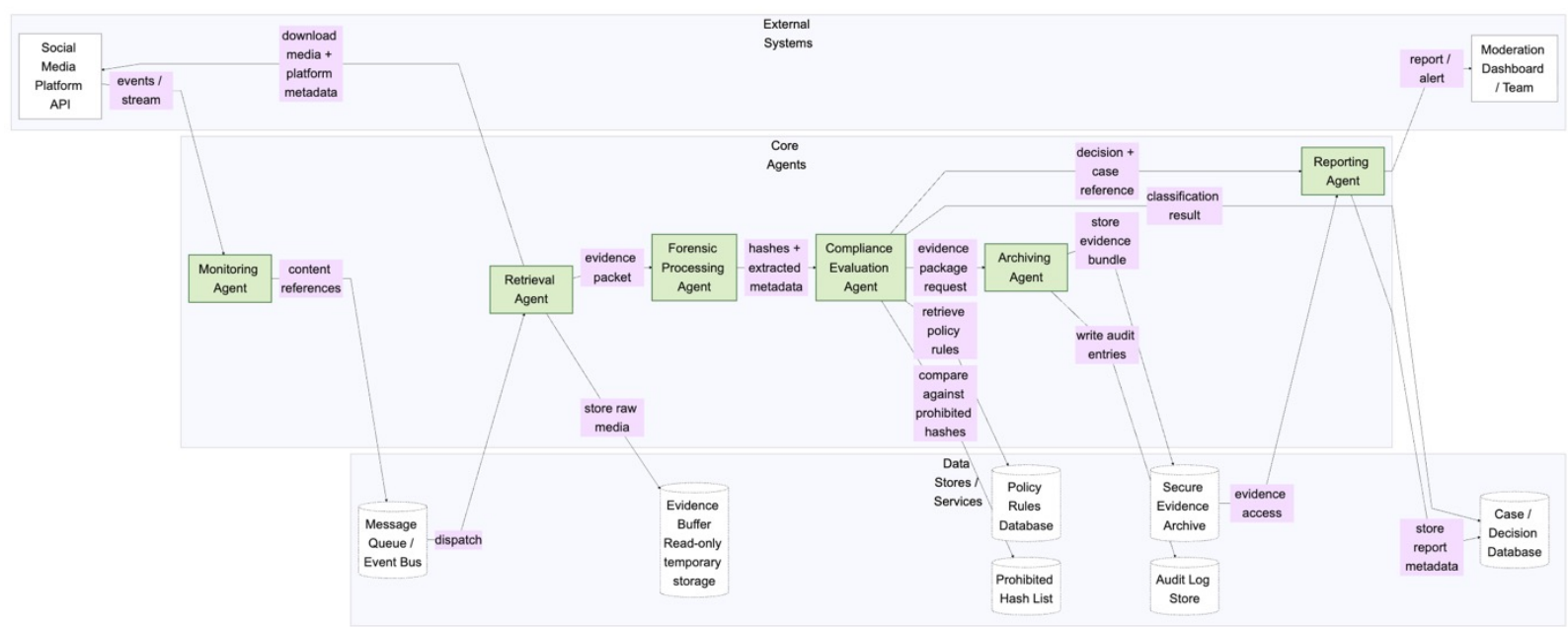
- The system must primarily be composed of autonomous, automated software agents.
- Those agents must be able to work independently of each other.
- Those agents must also be able to work in coordination with each other, as needed.
- The system must be able to apply business rules to reviewed content to label it as Allowed, Banned, or Pending with a greater than 95% accuracy rate.
- The system must provide a mechanism for human review of automated labelling outcomes.
- The system must store logs of all relevant activity to a database.



II. System Architecture

A. Full System Architecture

The full system architecture is composed of numerous independent, automated, cloud-based agents.

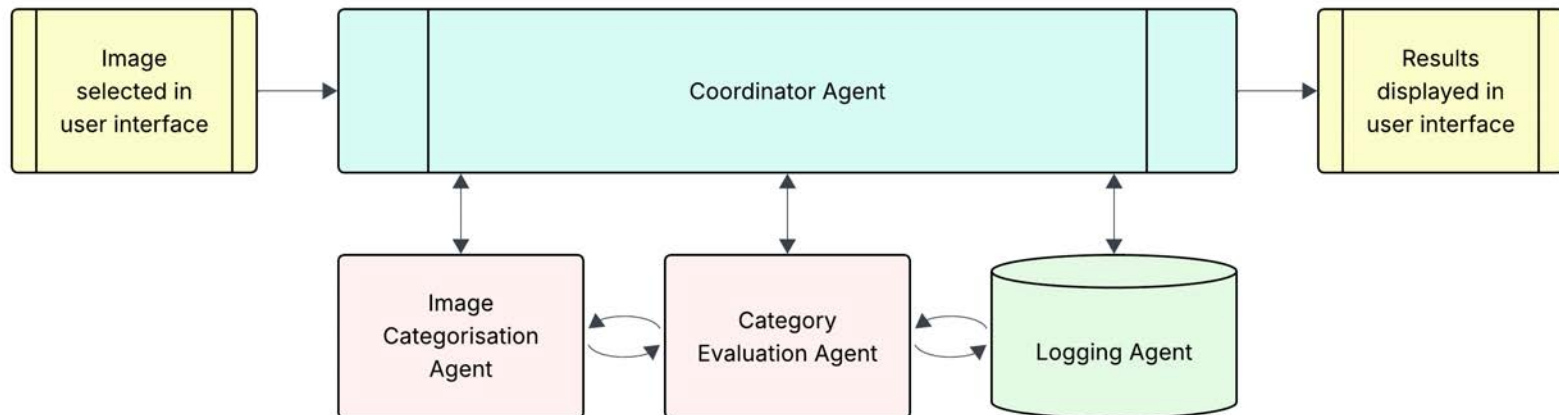




II. System Architecture

B. Implemented Extract of the Full System Architecture

The Coordinator Agent acts as a dispatcher or controller agent that assigns tasks to, and monitors the results of processing undertaken by, the Categorisation, Evaluation, and Logging Agents.





II. System Architecture

C. Design Principles

1. Tightly Focused Scope of Responsibility

Each agent must focus on a single task and only hold responsibility over that same task.

2. Agnostic to Other Agents

Each agent must be able to operate without any knowledge of the inner workings of the other agents.

3. Cloud-Based Runtime

Each agent must operate in a cloud-based runtime, absent a compelling reason to the contrary.

4. Duplicable & Scalable

Each agent must be able to be duplicated on demand, with runtime resources scaled accordingly.



III. Agent Components

A. Coordinator Agent

This agent acts as a dispatcher that coordinates the actions of the other agents in sequence. It also returns the results of image processing to the live demo front end user interface.

B. Categorisation Agent

This agent processes a supplied image through the AWS Rekognition image categorisation service, then returns the results to the coordinator agent.

C. Evaluation Agent

This agent evaluates the image category, as determined by the categorisation agent, against business rules for accepting or rejecting the image for publication, then returns results to the coordinator agent.

D. Logging Agent

This agent records the processing steps and the result of image processing in a MongoDB database.



III. A. Coordinator Agent

```
def processImage(image):
    print(f'processImage() - image: {image}')

    categorization = runLambda(
        os.environ['AGENT_CATEGORIZATION'],
        {'case': image}
    )
    print(f'processImage() - categorization: {json.dumps(categorization, indent=4)}')

    evaluation = runLambda(
        os.environ['AGENT_EVALUATION'],
        categorization
    )
    print(f'processImage() - evaluation: {json.dumps(evaluation, indent=4)}')
```



III. B. Categorisation Agent

```
def handler(event, context):
    print(f'event: {json.dumps(event, indent=4)}')

    categorizationCase = f'{event["case"].lower()}.jpg'
    print(f'categorizationCase: {categorizationCase}')

    categorizationLabels = detectImageLabels(categorizationCase)
    print(f'categorizationLabels: {json.dumps(categorizationLabels, indent=4, default=str)}')

    return categorizationLabels

def detectImageLabels(imageFile):
    return rekognitionClient.detect_labels(
        Image={
            'S3Object': {
                'Bucket': os.environ['S3_INGEST_BUCKET'],
                'Name': imageFile
            }
        },
        MaxLabels=1,
        MinConfidence=95
    )
```




III. B. Categorisation Agent Results

```
"LabelModelVersion": "3.0",
"Labels": [
  {
    "Aliases": [],
    "Categories": [
      {
        "Name": "Vehicles and Automotive"
      }
    ],
    "Confidence": 99.48323822021484,
    "Instances": [],
    "Name": "Pickup Truck",
    "Parents": [
      {
        "Name": "Transportation"
      },
      {
        "Name": "Truck"
      },
      {
        "Name": "Vehicle"
      }
    ]
  }
]
```



III. C. Evaluation Agent

```
evaluationRules = {
    'apple': {
        'review': False,
        'status': 'banned',
        'type': 'food'
    },
    'car': {
        'review': False,
        'status': 'allowed',
        'type': 'vehicle'
    },
    'pickup truck': {
        'review': True,
        'status': 'pending',
        'type': 'vehicle'
    },
    'truck': {
        'review': True,
        'status': 'pending',
        'type': 'vehicle'
    }
}
```

```
def handler(event, context):
    print(f'event: {json.dumps(event, indent=4)}')

    evaluationMaterial = json.loads(event)

    subjectLabel = evaluationMaterial['Labels'][0]['Name'].lower()
    print(f'subjectLabel: {subjectLabel}')

    evaluationResult = evaluationRules.get(subjectLabel)
    print(f'evaluationResult: {evaluationResult}')

    if evaluationResult:
        return evaluationResult

    return {
        'review': False,
        'status': 'banned',
        'type': 'unknown'
    }
```



III. D. Logging Agent

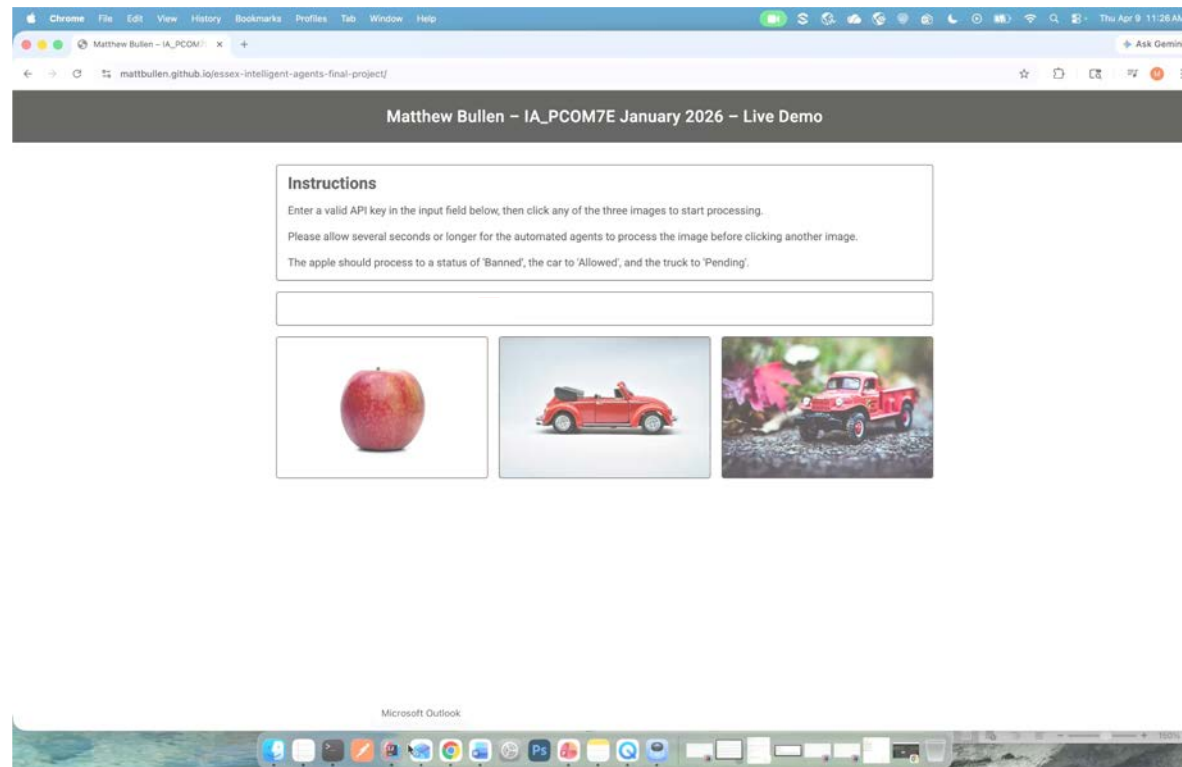
```
def handler(event, context):
    print(f'event: {json.dumps(event, indent=4)}')

    checkForRecord = list(mongoCollection.find({'image': event['image']}))
    print(f'checkForRecord: {json.dumps(checkForRecord, indent=4, default=str)}')

    if not checkForRecord:
        createRecord = mongoCollection.insert_one(event)
        print(f'logActivity() - createRecord: {event["image"]} | {json.dumps(str(createRecord.inserted_id), indent=4)}')
        return {'inserted_id': str(createRecord.inserted_id)}

    updateRecord = mongoCollection.update_one({'image': event['image']}, {'$set': event})
    print(f'logActivity() - updateRecord: {event["image"]} | {json.dumps(str(updateRecord.raw_result), indent=4)}')
    return {'matched_count': updateRecord.matched_count,
            'modified_count': updateRecord.modified_count}
```

IV. Live Demo Video



Note: This slide is provided for convenience and as a walk-through of the live demo. The API key needed to access the live demo is provided in a separate file.